

Cairo University

Faculty of Computers and Artificial Intelligence



CS251

Introduction to

Software Engineering

Invest Wise

Software Design Specifications

Version 1.0

May 2025



CS251: MG

Project: Invest Wise

Software Design Specification

Contents

Team	3
Document Purpose and Audience	3
Purpose:	3
Audience:	3
System Models	3
I. Architecture Diagram	3
II. Class Diagram(s)	6
III. Class Descriptions	7
IV. Sequence diagrams	10
Class - Sequence Usage Table	13
V. State Diagram	16
VI. SOLID Principles	17
VII. Design Patterns	18
Tools	18
Ownership Report	19



CS251: **MG**

Project: **Invest Wise**

Software Design Specification

Team

ID	Name	Email	Mobile
20230620	Mohamed Mahmoud Bayoumi	mohammed.elgabry7@gmail.com	01205657495
20231041	George Ezat Hosni	george.ezat1@gmail.com	
20231229	Maria Atef Edward	mariaatef517@gmail.com	

Document Purpose and Audience

Purpose:

This document outlines the design of InvestWise, a Sharia-compliant investment app, including architecture, class diagrams, and design patterns.

Audience:

- Developers
- QA engineers
- Stakeholders (Egyptian investors, Islamic finance experts)
- Project managers.

System Models

I. Architecture Diagram

Software Architecture: Micro-Service Architecture

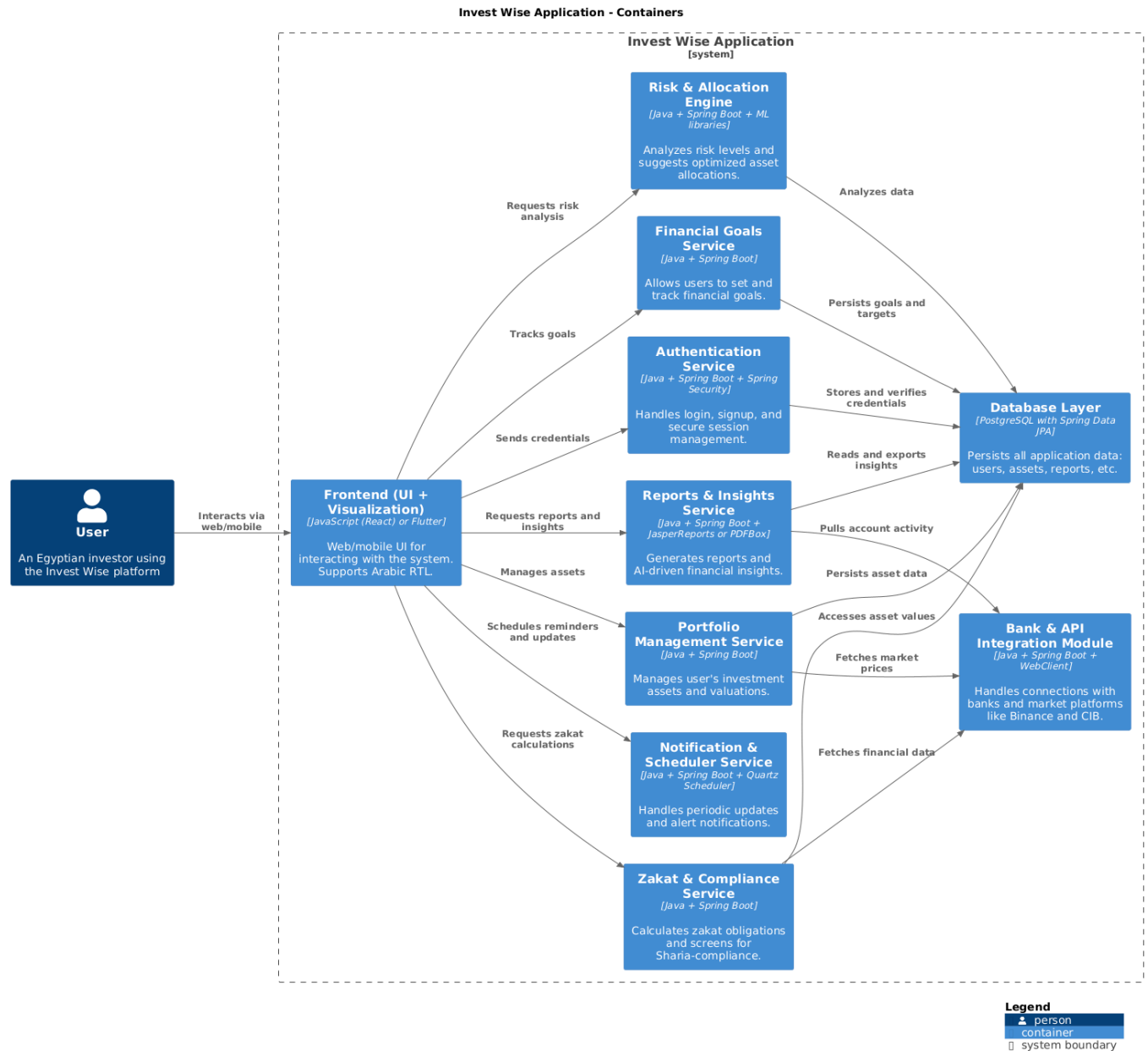
- This microservices-based architecture was chosen for its scalability, modularity, and ability to support real-time financial services. It allows each component such as risk analysis, portfolio management, and compliance to operate independently, making the system easier to maintain and extend. The inclusion of Arabic RTL support and zakat calculation ensures cultural and regional relevance, especially for users in the Middle East and North Africa. Integration with financial APIs and AI-driven insights also makes it a powerful and intelligent investment platform



CS251: **MG**

Project: **Invest Wise**

Software Design Specification





CS251: **MG**
Project: **Invest Wise**

Software Design Specification

Component	Responsibilities	Tech Stack
Frontend UI	Handles all user interactions, visualizations, RTL Arabic support	React/Flutter
Authentication Service	Manages login, signup, secure sessions	Java, Spring Boot, Spring Security
Portfolio Management System	Manages investment asset CRUD operations	Java/Spring Boot
Zakat & Compliance Service	Calculates zakat, applies halal filters	Java, Spring Boot
Reports & Insights Service	Generates reports, provides AI insights	Java, Spring Boot + Jasper Report
Financial Goals Service	Allows setting and tracking investment goals	Java, Spring Boot
Risk & Allocation Engine	Performs risk analysis and asset rebalancing	Java, Spring Boot, Web Client
Bank & API Integration Module	Integrates with financial institutions and platforms(e.g., Binance, CIB)	Java, Spring Boot, Web Client
Notification & Scheduler Service	Sends reminders and periodic alerts	Java, Spring Boot + Quartz
Database layer	Stores user data,goals,assets and reports	PostgreSQL + Spring Data JPA

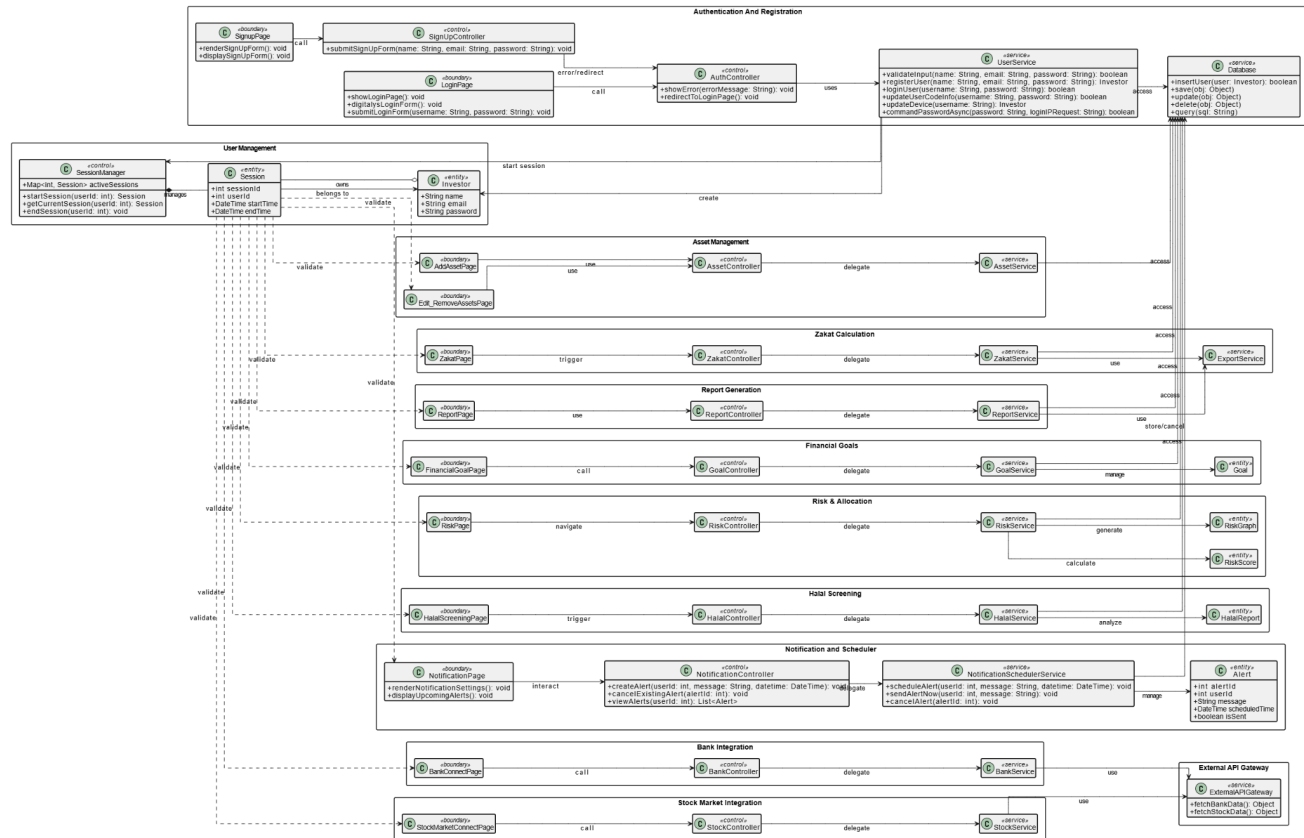


CS251: **MG**

Project: **Invest Wise**

Software Design Specification

Full Class Diagram



III. Class Descriptions

Class ID	Class Name	Description & Responsibility
1.	Investor	Represents the system's user (investor). Holds personal credentials and acts as the entry point to the system.
2.	SessionManager	Handles the lifecycle of a session including starting, retrieving, and ending sessions for authenticated users across pages.
3.	Session	Represents the data structure of an active session, including user ID, session ID, and session timestamps.
4.	SignupPage	Renders and displays the registration form for new users and facilitates input capture during signup.
5.	LoginPage	Provides the user interface for login, capturing credentials and initiating authentication.



CS251: **MG**

Project: **Invest Wise**

Software Design Specification

Class ID	Class Name	Description & Responsibility
6.	SignUpController	Processes user registration data from the signup form and sends it to the AuthController for validation.
7.	AuthController	Controls the flow of authentication. Redirects users, shows errors, and delegates validation to UserService.
8.	UserService	Manages user-related business logic such as input validation, registration, login handling, and device recognition. It also serves as a bridge between the controllers and the data layer.
9.	AddAssetPage	Presents a form UI for users to input new assets and interacts with the controller to handle submission.
10.	Edit_RemoveAssetsPage	Displays a list of user assets, providing interfaces to edit or delete selected entries
11.	AssetController	Coordinates actions from the UI with business logic in AssetService for asset creation, update, or deletion
12.	AssetService	Encapsulates core asset-related logic: validation, CRUD operations, asset filtering, and custom asset functions such as adding Accenture assets.
13.	ZakatPage	Provides the interface to trigger zakat calculation and download zakat reports.
14.	ZakatController	Manages user requests to compute zakat and delegates the business logic to ZakatService.
15.	ZakatService	Calculates zakat, retrieves relevant user asset data, and prepares summaries for reporting.
16.	ReportPage	Interface for generating and exporting financial reports based on user data.
17.	ReportController	Receives report generation requests and sends them to ReportService for processing and export.
18.	ReportService	Handles investment data aggregation, report formatting, and coordinates export functionality via ExportService.
19.	ExportService	Provides file export features for zakat summaries and financial reports in various formats (PDF, etc.).
20.	Database	Simulates the data layer for all persistent operations including insert, update, delete, and complex queries.
21.	FinancialGoalPage	Displays a form for users to define new financial goals and view existing goals like savings targets or investment objectives.
22.	GoalController	Handles interactions from the FinancialGoalPage, processes goal submission or listing requests, and delegates business logic to GoalService.
23.	GoalService	Manages the validation, storage, and retrieval of user financial goals. Ensures goals are properly structured and tracked.
24.	Goal	Represents an individual financial goal, holding information like type, target amount, deadline, and current progress
25.	RiskPage	Provides the user interface to initiate a risk assessment, view graphical risk representations, and view mitigation advice



CS251: **MG**

Project: **Invest Wise**

Software Design Specification

Class ID	Class Name	Description & Responsibility
26.	RiskController	Processes user requests for risk analysis, graphical visualization, and recommended actions. Delegates risk computations to RiskService.
27.	RiskService	Performs portfolio-based risk analysis, generates risk scores, constructs graphical data for insights, and suggests mitigation strategies
28.	RiskScore	Encapsulates a user's portfolio risk score and categorizes the risk level (e.g., Low, Medium, High)
29.	RiskGraph	Represents the data structure for visual graph outputs related to portfolio risk (e.g., pie charts, bar charts)
30.	HalalScreeningPage	Provides options for users to apply Halal investment filters, view filtered portfolios, and download Halal compliance reports
31.	HalalController	Handles user actions related to screening for Halal compliance and delegates analysis and filtering to HalalService
32.	HalalService	Analyzes investment portfolios to determine Halal compliance, filters non-compliant assets, and generates Halal screening reports
33.	HalalReport	Represents the outcome of a Halal screening, categorizing assets into Halal and non-Halal lists along with summary data.
34.	BankConnectPage	Provides a user interface for securely connecting a user's bank account to the platform for seamless transactions and tracking.
35.	BankController	Handles user attempts to link their bank accounts, processes OTP verifications, and delegates secure processing to BankService.
36.	BankService	Verifies bank card details, sends and confirms OTPs, and establishes a trusted connection between user bank accounts and the platform.
37.	StockMarketConnectPage	Interface for users to link their stock brokerage accounts to the platform to import investment data.
38.	StockController	Manages requests to connect to stock brokerage platforms, handles credential submission and validation.
39.	StockService	Verifies stock platform credentials, manages connection sessions, and links stock accounts to the user profile for tracking.
40.	ExternalAPIGateway	Acts as an intermediary for connecting with external financial data sources (e.g., stock market APIs, bank APIs). Provides methods like <code>fetchBankData()</code> and <code>fetchStockData()</code> to retrieve real-time external data.
41.	NotificationPage	Displays upcoming alerts and notifications to the user, providing an interface to view scheduled alerts.
42.	NotificationController	Manages user interactions for creating, viewing, and canceling notifications. Sends requests to NotificationSchedulerService
43.	NotificationSchedulerService	Handles scheduling, updating, and canceling of alerts. Ensures notifications are correctly timed and triggered.
44.	Alert	Represents an individual alert object, storing information like user ID, message content, scheduled time, and sent status



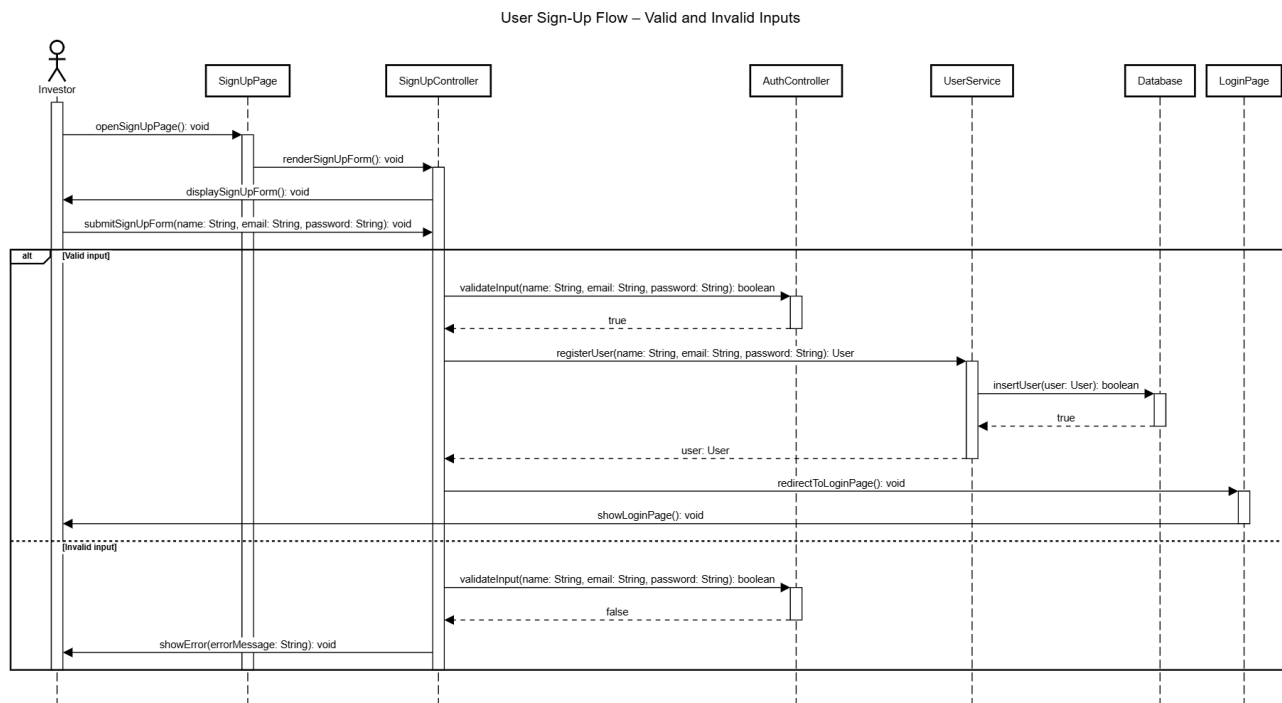
CS251: MG

Project: Invest Wise

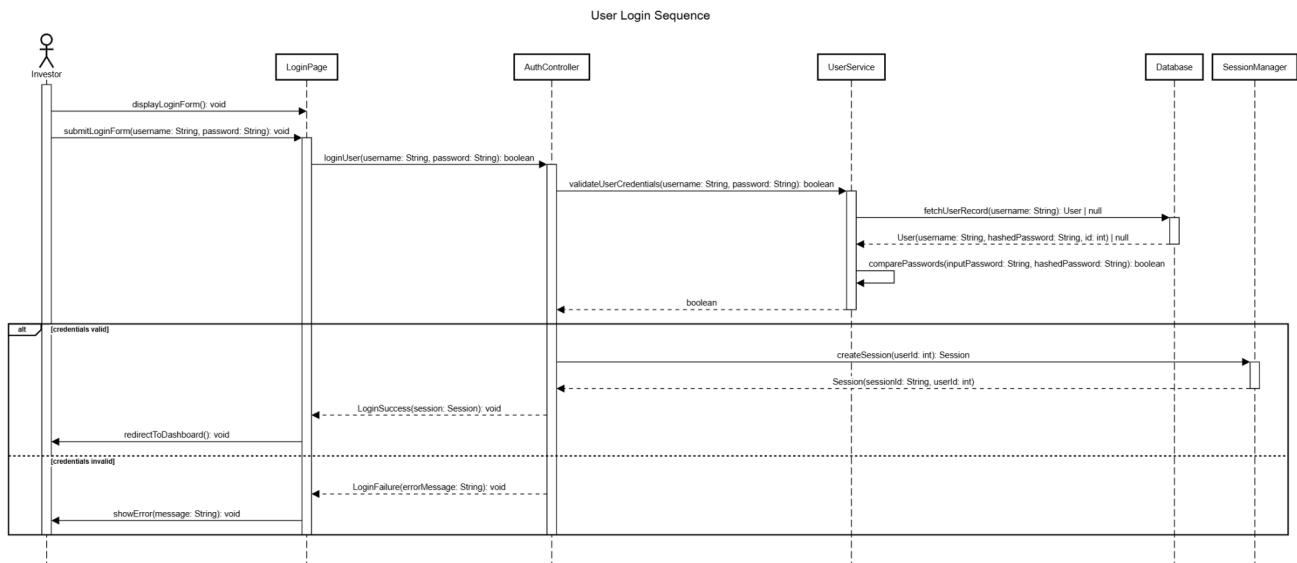
Software Design Specification

IV. Sequence diagrams

1. User SignUp



2. User Login



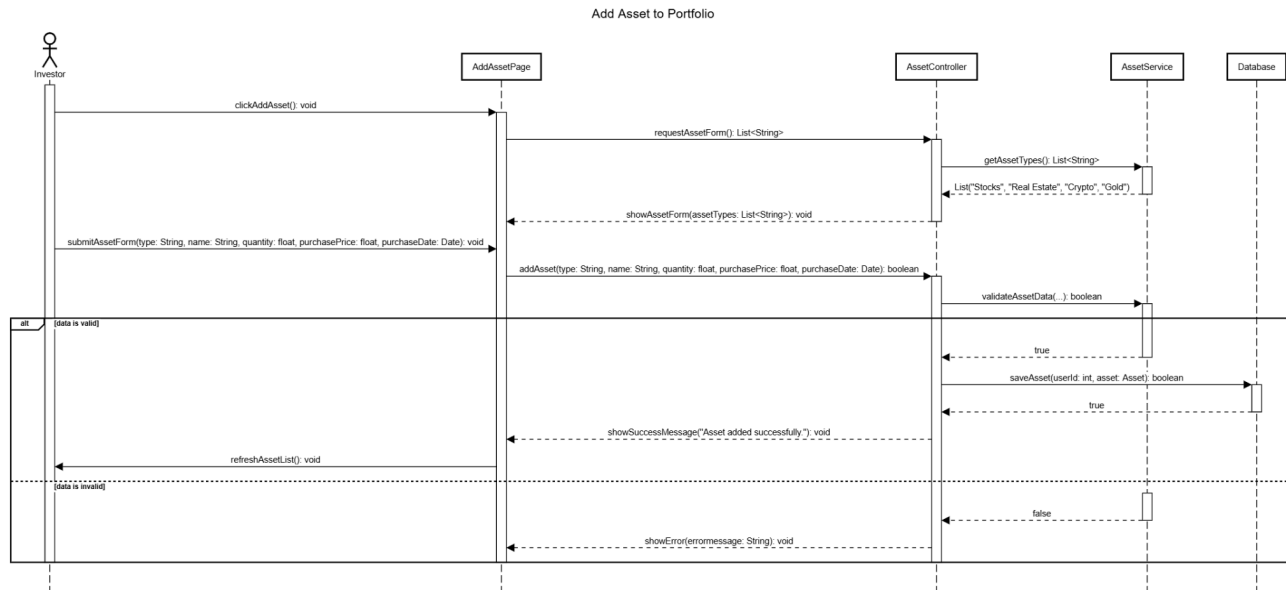


CS251: MG

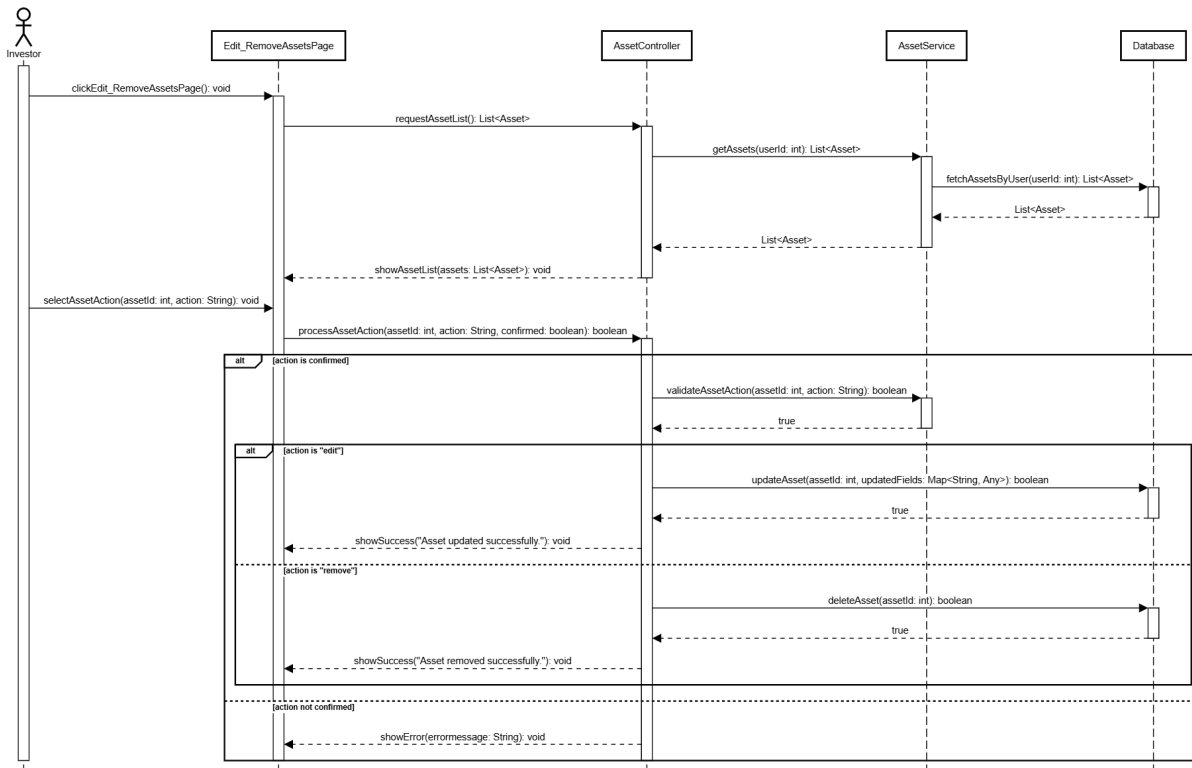
Project: Invest Wise

Software Design Specification

3. Add Assets



4. Edit/Remove Assets



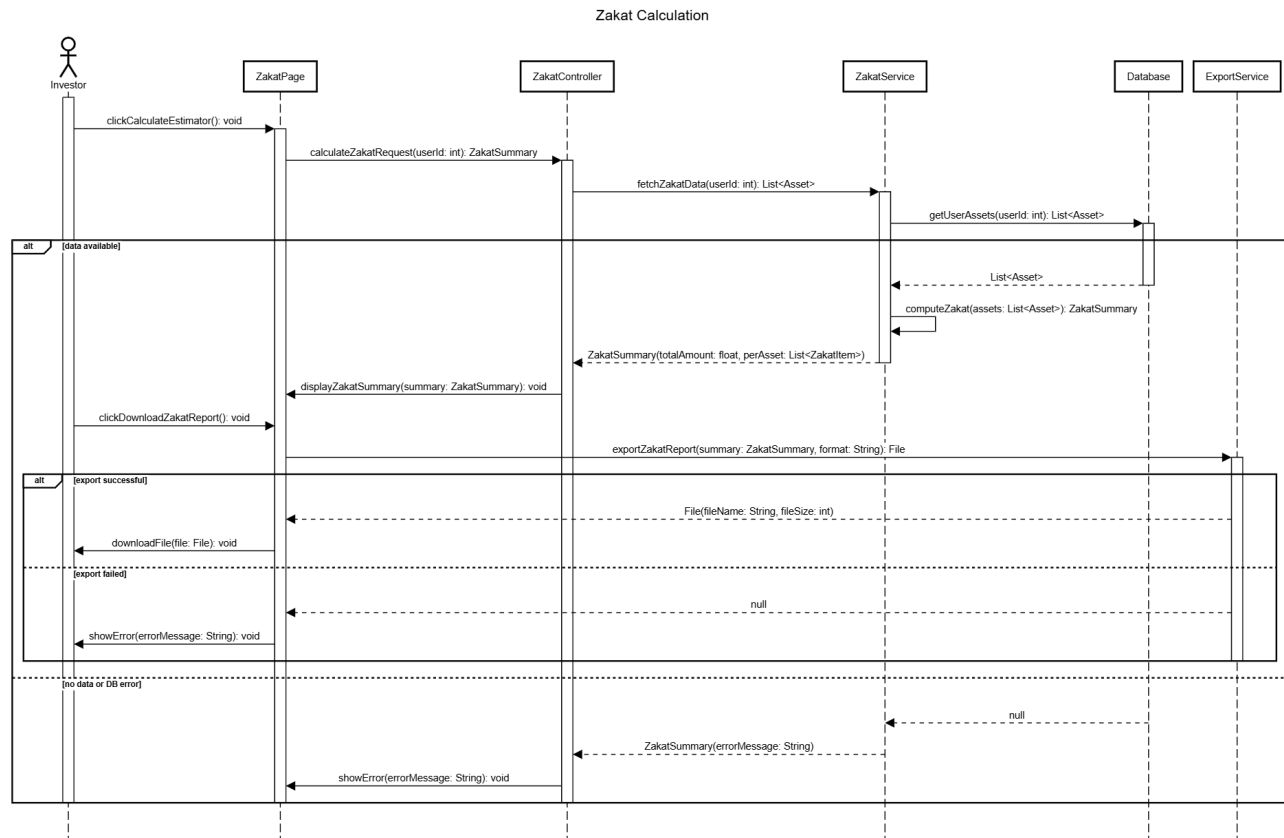


CS251: MG

Project: Invest Wise

Software Design Specification

5. Zakat Calculation



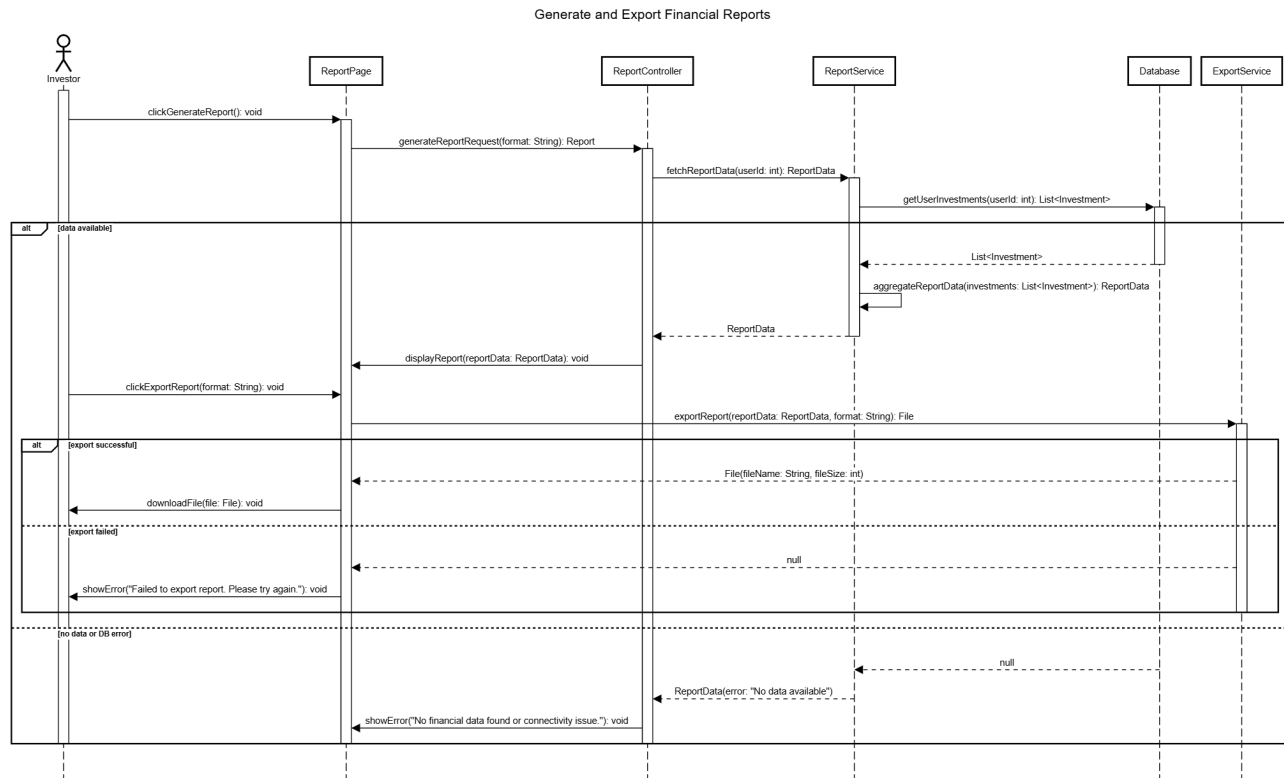


CS251: MG

Project: Invest Wise

Software Design Specification

6. Generate and Export Financial Reports



Class - Sequence Usage Table

Sequence Diagram	Classes Used	All Methods Used
User Sign Up	- Investor - SignupPage - SignUpController - AuthController - UserService - DataBase - LoginPage	- openSignUpPage(): void - renderSignUpForm(): void - displaySignUpForm(): void - submitSignUpForm(name: String, email: String, password: String): void - validateInput(name: String, email: String, password: String): boolean - registerUser(name: String, email: String, password: String): User - insertUser(user: User): boolean - redirectToLoginPage(): void - showLoginPage(): void - showError(errorMessage: String): void



CS251: **MG**

Project: **Invest Wise**

Software Design Specification

User Login	- Investor - LoginPage - AuthController - UserService - Database - SessionManager	- digitalysLoginForm(): void - submitLoginForm(username: String, password: String): void - loginUser(username: String, password: String) boolean - updateUserCodeInfo(username: String, password: String) boolean - updateDevice(username: String) User [null] - commandPasswordAsync(Password: String, loginIPRequest: String) boolean - redirectToDateResult() void - showEventPackage: Strings void - enableHeaderHandler int Session
Add Assets	- Investor - AddAssetPage - AssetController - AssetService - Database	- requestAssetForm(s) List<String> - showAssetForm(pastType: List<String>) val - addAssetType(type: String, name: String, quantity: final, purchasePrice: final, purchaseDate: Date) boolean - getAsset(Inputs) List<String> - validateAssetData(s): boolean - showAccenture(pastType: partial, purchasePrice: final, purchaseDate: Date) val - addAccenture(Outputs: String, void...)
Edit/Remove Assets	- Investor - Edit_RemoveAssetsPage - AssetController - AssetService - Database	- clickEdit_RemoveAssetsPage(): void - requestAssetList(): List<Asset> - getAssets(userId: int): List<Asset> - fetchAssetsByUser(userId: int): List<Asset> - showAssetList(assets: List<Asset>): void - selectAssetAction(assetId: int, action: String): void - processAssetAction(assetId: int, action: String, confirmed: boolean): boolean - validateAssetAction(assetId: int, action: String): boolean - updateAsset(assetId: int, updatedFields: Map<String, Any>): boolean - deleteAsset(assetId: int): boolean - showSuccess(message: String): void - showError(errorMessage: String): void



CS251: **MG**

Project: **Invest Wise**

Software Design Specification

Zakat Calculation	- Investor - ZakatPage - ZakatController - ZakatService - Database - ExportService	- clickCalculateEstimator(): void - calculateZakatRequest(userId: int): ZakatSummary - fetchZakatData(userId: int): List<Asset> - getUserAssets(userId: int): List<Asset> - computeZakat(assets: List<Asset>): ZakatSummary - displayZakatSummary(summary: ZakatSummary): void - clickDownloadZakatReport(): void - exportZakatReport(summary: ZakatSummary, format: String): File - downloadFile(file: File): void - showError(errorMessage: String): void
Generate and Export Financial Reports	- Investor - ReportPage - ReportController - ReportService - Database - ExportService	- clickGivenstaffReport(): void - generateReportRequestWarrant(): String → Report - feedReportDataSearch(int): ReportData - getUserInvestmentSubmit(int): List<Investment> - clickExportReportWarrant(): String → void - displayReport(reportData: ReportData): void - aggregateReportData(Investments: List<Investment>): ReportData - exportReport(reportData: ReportData, format: String): File - downloadFile(file: File): void - showError("Failed to export report. Please try again."): void - ShowError("No financial data found or connectivity issue."): void



CS251: MG

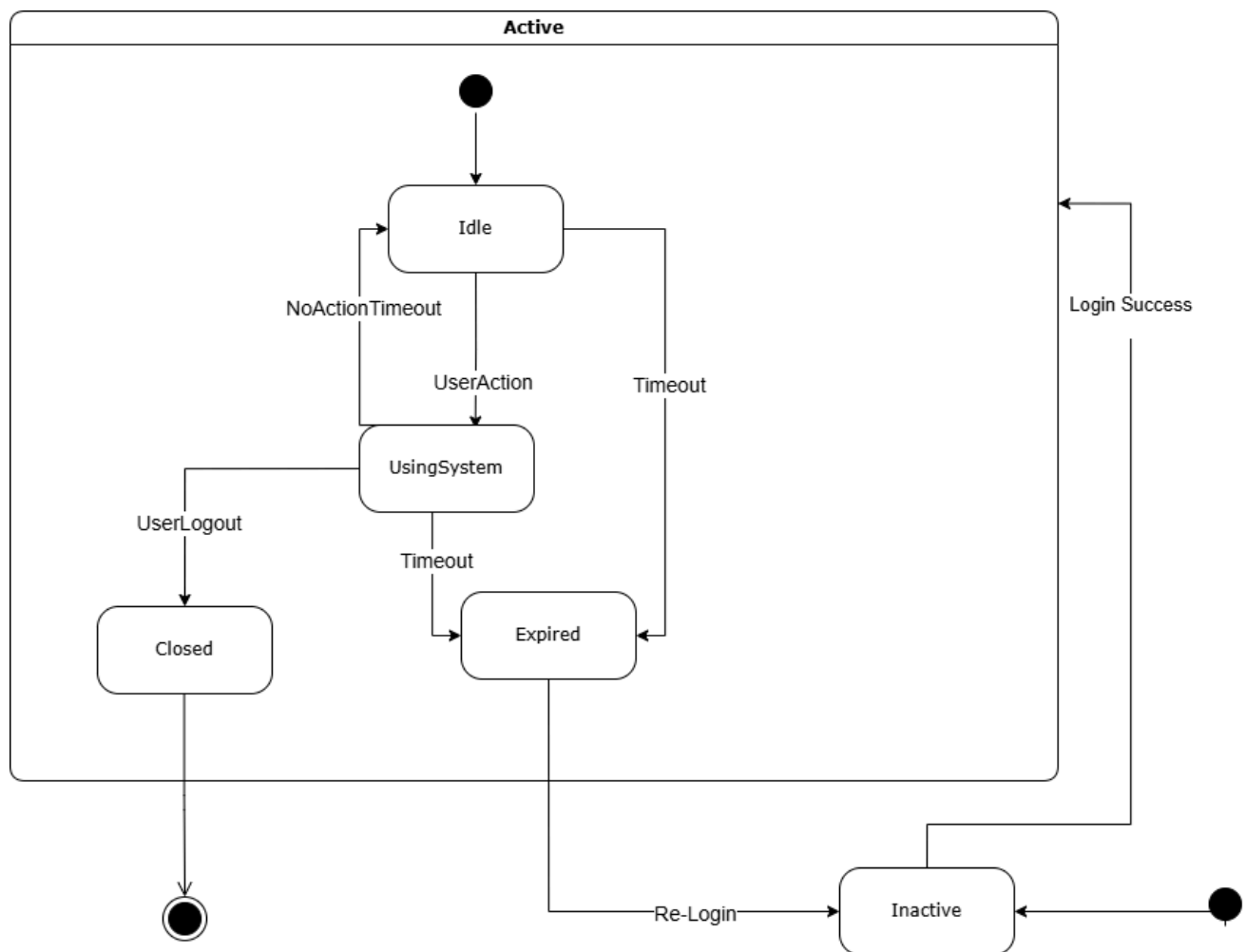
Project: Invest Wise

Software Design Specification

V. State Diagram

Object: Session

State Diagram:





CS251: **MG**

Project: **Invest Wise**

Software Design Specification

VI. SOLID Principles

SOLID Principles	How It Was Applied	Where
S - Single Responsibility Principle (SRP)	Each class has one clear purpose: • User holds user credentials and ID only • AssetController only manages assets • ZakatController only coordinates zakat-related tasks • ReportExportService handles export logic	User , AssetController , ZakatController , ReportExportService , AuthController
O - Open/Closed Principle (OCP)	System is open for extension but closed for modification: • New asset types added via AssetFactory • Interfaces (IAssetService, etc.) allow extending functionality without modifying core logic	AssetFactory , IAssetService , AssetController
L - Liskov Substitution Principle (LSP)	Interfaces allow replacing implementations without breaking functionality: • Any class implementing IAssetService or IReportExportable can be used interchangeably	IAssetService , IReportExportable
I - Interface Segregation Principle (ISP)	Each class or UI component uses only the methods it needs: • SignupPage uses only signup-related methods • LoginPage uses only login-related methods • No class is forced to implement unused methods	Class Diagram: SignupPage , LoginPage , AddAssetPage , Edit_RemoveAssetsPage
D - Dependency Inversion Principle (DIP)	High-level modules depend on abstractions, not on concrete implementations: • UserService and AssetService rely on Database interface • ZakatController uses ZakatCalculationFacade abstraction	UserService → Database , AssetService → Database , ZakatController → ZakatCalculationFacade



CS251: MG

Project: Invest Wise

Software Design Specification

VII. Design Patterns

Design Pattern	Where It Was Used	Why It Was Beneficial
Factory Pattern	<code>AssetFactory.java</code>	Centralized asset creation logic. Easily extensible to support new asset types without altering client code.
Facade Pattern	<code>ZakatCalculationFacade.java</code>	Simplifies zakat calculation by hiding complex logic behind a clean interface used by controllers.
Singleton Pattern	<code>ReportExportService.java</code> .	Ensures a single consistent instance for exporting reports. Promotes reuse and avoids duplication.

Tools

- structurizr.com
- sequencediagram.org
- draw.io



CS251: **MG**
Project: **Invest Wise**

Software Design Specification

Ownership Report

Owner	Items Created
Mohamed Mahmoud Bayoumi	- Contributor in Architecture Diagram - Contributor in Class Diagram - Contributor in Sequence Diagrams - Contributor in State Diagram - Contributor in the other things
George Ezat Hosni	- Contributor in Architecture Diagram - Contributor in Class Diagram - Contributor in Sequence Diagrams - Sequence Description Table - Contributor in the other things
Maria Atef Edward	- Contributor in Architecture Diagram - Contributor in Class Diagram - Class Description table - Contributor in Sequence Diagrams - Contributor in State Diagram - Contributor in the other things